

```
In [1]: # Behzad Ghabaei
# CS 82A - Data Science
# module3_lab2.ipynb
# Textbook Problem 2.1
# Outliers in the crime data
# Instructor Seno
# 9/16/2026
```

The dataset is called "usarrests.csv" and has statistics for arrests per 100,000 residents for assault and murder, among the 50 states in 1973. The percentage of the population living in urban areas is given. UrbanPop is a percentage out of 100. None should exceed 100%.

```
In [2]: import pandas as pd
df = pd.read_csv("usarrests.csv")
```

Step 1: Load the dataset and size up the file

The four commands you should always run first are: **df.head()** **df.shape** **df.info()** **df.describe()**. Since we have a small dataset we can use head() to see all of our data.

```
In [3]: df.head(10)
```

```
Out[3]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13.2	236.0	58
1	Alaska	10.0	263.0	48
2	Arizona	8.1	294.0	80
3	Arkansas	8.8	190.0	50
4	California	9.0	276.0	91
5	Colorado	7.9	204.0	78
6	Connecticut	3.3	110.0	77
7	Delaware	5.9	238.0	72
8	Florida	15.4	335.0	80
9	Georgia	17.4	NaN	60

At first glance, we see our data is missing a value on line 9, Georgia's "Assault" category, we also see an UrbanPop at 570 for Iowa. Next, we see South Carolina at 879 for Assault. The data types are floating numbers, New York's UrbanPop looks too low at 6, this is the only single digit number. This seems to be more of a 60 than 6.

The shape() confirms the size. 50 rows , 4 columns

```
In [4]: df.shape
```

```
Out[4]: (50, 4)
```

The data types and missing values are caught here. Assault shows us 49/50

```
In [5]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0    50 non-null     str
1   Murder        50 non-null     float64
2   Assault       49 non-null     float64
3   UrbanPop      50 non-null     int64
dtypes: float64(2), int64(1), str(1)
memory usage: 1.7 KB
```

It is useful to know the max, min, mean and median, for filling missing values.

```
In [6]: df.describe()
```

```
Out[6]:
```

	Murder	Assault	UrbanPop
count	50.00000	49.000000	50.00000
mean	7.78800	182.183673	74.20000
std	4.35551	130.877435	73.40828
min	0.80000	45.000000	6.00000
25%	4.07500	109.000000	53.25000
50%	7.25000	159.000000	66.00000
75%	11.25000	249.000000	77.75000
max	17.40000	879.000000	570.00000

From describe() several outliers are visible such as UrbanPop min at 6 and max at 570. Assault min at 45 and max at 879, and Murder min at .8 and max at 17.4. We are mostly interested in changing 570 and 879.

What are the data types we're working with. Murder and Assault are float data types.

```
In [7]: df.dtypes
```

```
Out[7]: Unnamed: 0      str
Murder      float64
Assault      float64
UrbanPop     int64
dtype: object
```

We know the correct spelling of our columns

```
In [8]: df.columns
```

```
Out[8]: Index(['Unnamed: 0', 'Murder', 'Assault', 'UrbanPop'], dtype='str')
```

Step 2: Write your first observations

Looking at the data, many concerns arise about the precision of the data. First, using floats represents the rate per 100,000, urban populations should not exceed 100%. The urban population max is at 570. min is at 6. For assault, the max is at 879, min is at 45. For murder, max is at 17.4 min is at 0.8.

Assault is missing a value, that should be filled.

Step 3: Count the missing values

```
In [9]: # apply tutorial 1 to the whole dataframe
df.isna().sum() # 1 missing value under assault
```

```
Out[9]: Unnamed: 0    0
Murder      0
Assault     1
UrbanPop    0
dtype: int64
```

Step 4: Impute the missing value, dropping rows loses valuable data.

```
In [10]: df.shape # run this before and after filling missing values.
```

```
Out[10]: (50, 4)
```

```
In [11]: # Fill the missing prices using the Tutorial 2 pattern
# Looking for: Impossible Values, Suspicious Outliers, Missing Values, Floating Point Noise
fill_value = df["Assault"].median() # The 50% median of "Assault" when running df.describe() is 159.
# whether mean or median fits your lab data better
df["Assault"] = df["Assault"].fillna(fill_value)
print(df.head(20))
```

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13.2	236.0	58
1	Alaska	10.0	263.0	48
2	Arizona	8.1	294.0	80
3	Arkansas	8.8	190.0	50
4	California	9.0	276.0	91
5	Colorado	7.9	204.0	78
6	Connecticut	3.3	110.0	77
7	Delaware	5.9	238.0	72
8	Florida	15.4	335.0	80
9	Georgia	17.4	159.0	60
10	Hawaii	5.3	46.0	83
11	Idaho	2.6	120.0	54
12	Illinois	10.4	249.0	83
13	Indiana	7.2	113.0	65
14	Iowa	2.2	56.0	570
15	Kansas	6.0	115.0	66
16	Kentucky	9.7	109.0	52
17	Louisiana	15.4	249.0	66
18	Maine	2.1	83.0	51
19	Maryland	11.3	300.0	67

```
In [12]: df.shape
```

```
Out[12]: (50, 4)
```

```
In [13]: df.isnull().sum()
```

```
Out[13]: Unnamed: 0    0
Murder      0
Assault     0
UrbanPop    0
dtype: int64
```

Assault no longer has a null value. The 50 percentile replaced the NaN.

Step 5: Remove duplicate rows

```
In [14]: print(df.duplicated().sum()) # counts fully repeated rows
```

0

No duplicates are detected.

Step 6: Adjust the "max" of Assault at 879 and the max of Urban Pop at 570,

South Carolina Assault is an outlier that can be adjusted by moving forward one decimal place. Sometimes data is skewed due to decimal point issues. It might be a good idea to adjust this number. We could change it from 879 to 87.9 or impute with the median at 157.5

```
In [15]: df[df["Assault"] == 879]
```

```
Out[15]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
39	South Carolina	14.4	879.0	48

```
In [16]: # South Carolina changed to 87.9
df.loc[39, "Assault"] = 87.9
```

```
In [17]: df.describe()
```

```
Out[17]:
```

	Murder	Assault	UrbanPop
count	50.000000	50.000000	50.000000
mean	7.788000	165.898000	74.200000
std	4.355511	82.412612	73.408280
min	0.800000	45.000000	6.000000
25%	4.075000	106.750000	53.250000
50%	7.250000	157.500000	66.000000
75%	11.250000	246.250000	77.750000
max	17.400000	337.000000	570.000000

```
In [18]: df.head(10)
```

```
Out[18]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13.2	236.0	58
1	Alaska	10.0	263.0	48
2	Arizona	8.1	294.0	80
3	Arkansas	8.8	190.0	50
4	California	9.0	276.0	91
5	Colorado	7.9	204.0	78
6	Connecticut	3.3	110.0	77
7	Delaware	5.9	238.0	72
8	Florida	15.4	335.0	80
9	Georgia	17.4	159.0	60

Looking at the data, it does not make sense that densely populated states have low populations such as New York at 6, and Maine at 51. What can be done is replace a value. For example Iowa is 570 but might need to be at 57, and perhaps New York should be at 60, instead of 6. Lets change these 2 values. The Pandas.pydata.org "Cookbook" snippet helps us by showing how to replace values. (https://pandas.pydata.org/docs/user_guide/cookbook.html)

```
In [7]: df.loc[df.AAA < 5, ["BBB", "CCC"]] = 2000
```

```
In [19]: # New York changed from 6 to 60.0 %
df.loc[31, "UrbanPop"] = 60
```

```
In [20]: # Iowa changed from 570 to 57 %
df.loc[14, "UrbanPop"] = 57
```

These represent numbers of Murder or Assault per 100,000 residents in 1973.

```
In [21]: df.head(10)
```

```
Out[21]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13.2	236.0	58
1	Alaska	10.0	263.0	48
2	Arizona	8.1	294.0	80
3	Arkansas	8.8	190.0	50
4	California	9.0	276.0	91
5	Colorado	7.9	204.0	78
6	Connecticut	3.3	110.0	77
7	Delaware	5.9	238.0	72
8	Florida	15.4	335.0	80
9	Georgia	17.4	159.0	60

Now lets take a look at describe for details. The count is the same, but the UrbanPop average has changed from 74 to 65. The min has changed from (New York 6) to (Vermont 32). The max has changed from (Iowa 570 to California 91). Has the median changed? No, it's still at 66.

```
In [22]: df.describe()
```

```
Out[22]:
```

	Murder	Assault	UrbanPop
count	50.000000	50.000000	50.000000
mean	7.78800	165.898000	65.020000
std	4.35551	82.412612	14.188944
min	0.80000	45.000000	32.000000
25%	4.07500	106.750000	54.500000
50%	7.25000	157.500000	66.000000
75%	11.25000	246.250000	76.500000
max	17.40000	337.000000	91.000000

```
In [23]: df [df["UrbanPop"] == 32] # this was the new min
```

```
Out[23]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
44	Vermont	2.2	48.0	32

```
In [24]: df [ df["UrbanPop"] == 91] # this was the new max
```

```
Out[24]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
4	California	9.0	276.0	91

None of the UrbanPop values exceed 100.

```
In [25]: df [ df["UrbanPop"] > 100 ]
```

```
Out[25]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
--	------------	--------	---------	----------

We have succesfully pre-processed the data. First adresssing all the missing values. Second, looking for outliers and noisy data to smooth, with data type conversion and min, max, adjustments, and with percentage cut off at 100. Lastly, lets establish a relation between urban population category and crime type.

- Small < 50%
- medium < 60%
- large < 70%
- extra-large >= 70%

We will check pandas.pydata.org for suggestions on how to categorize datasets. "User Guide" > "Table Visualization" > here is the snippet of a function with the if-elif-else condition:

```
[2]:
def rain_condition(v):
    if v < 1.75:
        return "Dry"
    elif v < 2.75:
        return "Rain"
    return "Heavy Rain"

styler.format(rain_condition)
```

(https://pandas.pydata.org/docs/user_guide/style.html)

```
In [26]: def UrbanPop_condition(v):
        if v < 50:
            return "Small"
        elif v < 60:
            return "Medium"
        elif v < 70:
            return "Large"
        else:
            return "Extra-Large"
        # the apply function was needed here
df['UrbanPop_Category'] = df["UrbanPop"].apply(UrbanPop_condition)
df.head(10)
```

```
Out[26]:
```

	Unnamed: 0	Murder	Assault	UrbanPop	UrbanPop_Category
0	Alabama	13.2	236.0	58	Medium
1	Alaska	10.0	263.0	48	Small
2	Arizona	8.1	294.0	80	Extra-Large
3	Arkansas	8.8	190.0	50	Medium
4	California	9.0	276.0	91	Extra-Large
5	Colorado	7.9	204.0	78	Extra-Large
6	Connecticut	3.3	110.0	77	Extra-Large
7	Delaware	5.9	238.0	72	Extra-Large
8	Florida	15.4	335.0	80	Extra-Large
9	Georgia	17.4	159.0	60	Large

Step 7: Save the cleaned file

A new csv file has been created with our Data Pre Processing with Urban Population categories and cut offs.

1. Data Cleaning a. Smooth noisy data b. Handling missing data c. Data Wrangling
2. Data Integration
3. Data Transformation
4. Data Reduction
5. Data Discretization (we changed the UrbanPop to categories: small, medium, large, and extra-large)

```
In [27]: # without the index column.  
# Hint: to_csv has an index= option.  
# Save the cleaned DataFrame to a CSV file  
df.to_csv('UrbanPop_Categories.csv', index=False)
```

Cleaning Log

The lab textbook problem 2.1 was completed in 6 - 7 steps.

In step 1. the dataset was loaded.

In step 2. Observations are recorded at first glance.

In step 3. Count the missing value

In step 4. Impute the missing value (median replaced NaN on "Assault")

In step 5. Remove duplicate rows

In step 6.

South Carolina Assault changed from 879 to 87.9

New York changed from 6 to 60.0

Iowa changed from 570 to 57

We established a relation between urban population category and crime type with a function `UrbanPop_condition()` containing an if-elif-else conditional.

End Of Lab - Behzad Ghabaei

```
In [ ]:
```